

Санкт-Петербургский государственный университет

Кафедра информатики

Группа 23.Б16-мм

Сравнение современных алгоритмов ОПТИМИЗАЦИИ

Колпащиков Максим Евгеньевич

Отчёт по курсовой работе
в форме «Сравнение»

Научный руководитель:
асс. каф. информатики Михайлов Д. А.

Санкт-Петербург
2025

Оглавление

Введение	3
1. Постановка задачи	5
2. Обзор	6
2.1. Обзор существующих решений: алгоритмы оптимизации	6
2.2. Обзор используемых технологий	8
3. Описание решения	11
3.1. Реализация программных модулей	11
4. Экспериментальное исследование	13
4.1. Условия проведения экспериментов	13
4.2. Исследовательские вопросы	14
4.3. Метрики оценки	14
4.4. Результаты экспериментов	16
4.5. Ответы на исследовательские вопросы и обсуждение ре- зультатов	45
4.6. Обсуждение результатов	47
4.7. Угрозы нарушения корректности	48
4.8. Воспроизводимость эксперимента	49
Заключение	51
Список литературы	54

Введение

Оптимизация является одной из фундаментальных задач, пронизывающей практически все сферы человеческой деятельности, от науки и инженерии до экономики и повседневной жизни. В самом общем смысле, оптимизация — это процесс нахождения наилучшего (оптимального) решения из некоторого множества доступных альтернатив при заданных критериях и ограничениях. Целью может быть максимизация прибыли, минимизация затрат, повышение эффективности системы, улучшение точности модели и многое другое.

Формально, задача оптимизации заключается в нахождении вектора переменных $\mathbf{x}^* \in D$, который минимизирует (или максимизирует) некоторую целевую функцию $f(\mathbf{x})$:

$$f(\mathbf{x}^*) = \min_{\mathbf{x} \in D} f(\mathbf{x})$$

где D — это область допустимых решений, определяемая набором ограничений (равенств или неравенств).

Для решения таких задач были разработаны многочисленные алгоритмы оптимизации, или оптимизаторы. Оптимизаторы — это итерационные вычислительные методы, предназначенные для систематического поиска оптимальных или близких к оптимальным решений. Они используются в тех случаях, когда:

- аналитическое решение задачи невозможно или слишком трудоемко;
- пространство поиска решений велико и сложно для прямого перебора;
- необходимо автоматизировать процесс принятия решений и настройки параметров сложных систем.

Примерами применения оптимизаторов служат обучение моделей машинного обучения (например, настройка весов нейронных сетей), проектирование инженерных конструкций, планирование логистических

маршрутов, управление финансовыми портфелями, калибровка научных приборов и многие другие.

Современный набор оптимизационных методов чрезвычайно широк и разнообразен: от классических градиентных методов (например, градиентный спуск, метод сопряженных градиентов) до более сложных стохастических, эвристических и метаэвристических алгоритмов (генетические алгоритмы, имитация отжига, методы роя частиц, алгоритмы дифференциальной эволюции и др.). Каждый из этих алгоритмов обладает своими сильными и слабыми сторонами, предположениями о характере целевой функции и области применимости.

В связи с таким многообразием возникает актуальная проблема выбора наиболее подходящего алгоритма для конкретного класса задач или даже для конкретной задачи. Эффективность алгоритма может зависеть от множества факторов: размерности задачи, наличия локальных экстремумов, гладкости целевой функции, типа ограничений, вычислительных ресурсов и требуемой точности. Данная работа посвящена исследованию и сопоставлению некоторых из таких современных алгоритмов.

1. Постановка задачи

Целью данной курсовой работы является проведение сравнительного анализа эффективности и особенностей применения современных алгоритмов оптимизации Adahessian [1], Adabelief [2], AdaBound [3], AdamW [8], Lion [15] при обучении нейронных сетей для решения задач регрессии, классификации и классификации изображений.

Для её выполнения были поставлены следующие задачи:

1. Провести краткий обзор теоретических основ и ключевых особенностей алгоритмов оптимизации.
2. Реализовать программные модули, необходимые для обучения нейросетевых моделей с использованием выбранных оптимизаторов, включая механизмы настройки гиперпараметров.
3. Провести экспериментальное обучение нескольких нейросетевых моделей на задачах регрессии, классификации и классификации изображений, применяя каждый из исследуемых оптимизаторов. Осуществить подбор оптимальных гиперпараметров для каждого оптимизатора и типа задачи.
4. Выполнить сравнительный анализ полученных результатов обучения, оценивая алгоритмы по таким метрикам, как скорость сходимости, достигнутое качество модели по различным метрикам, стабильность процесса обучения и обоснованность подбора гиперпараметров. Визуализировать основные результаты сравнения.

2. Обзор

Данный раздел посвящен обзору теоретических основ исследуемых алгоритмов оптимизации, а также описанию технологического стека, использованного для проведения экспериментов. Целью обзора является представление необходимой информации для понимания контекста работы, обоснование выбора конкретных методов и инструментов, а также краткое описание существующих подходов к оптимизации в области глубокого обучения. Поиск релевантной литературы осуществлялся с использованием научных поисковых систем Google Scholar и arXiv, с акцентом на оригинальные публикации, описывающие алгоритмы, и на документацию используемых программных библиотек.

2.1. Обзор существующих решений: алгоритмы оптимизации

Оптимизация параметров моделей глубокого обучения является ключевым этапом, определяющим их итоговую производительность. Современные исследования в этой области привели к созданию большого числа адаптивных алгоритмов оптимизации, стремящихся улучшить скорость сходимости и качество получаемых решений. В данной работе рассматриваются следующие алгоритмы:

- **AdamW** [8]. Является модификацией популярного алгоритма **Adam**, в которой исправлена проблема некорректного применения L2-регуляризации (**weight decay**). Вместо добавления L2-штрафа к функции потерь, **AdamW** применяет ослабление весов напрямую к весам модели на каждом шаге обновления, что, как утверждается, приводит к лучшей генерализации. Его преимуществом является сочетание адаптивности **Adam** с более корректным механизмом регуляризации, что делает его популярным выбором во многих современных архитектурах.
- **Lion** [15]. Алгоритм отличается простотой (обновляет веса, ис-

пользуя только операцию знака момента) и, по заявлениям авторов, демонстрирует высокую эффективность на задачах обучения больших моделей, в частности трансформеров, превосходя Adam по производительности при меньшем потреблении памяти. К потенциальным недостаткам можно отнести новизну и, возможно, меньшую изученность его поведения на широком спектре задач по сравнению с AdamW.

- **AdaBound** [3]. Данный алгоритм стремится объединить преимущества адаптивных методов (быстрая начальная сходимость) и SGD (хорошая генерализация). Он динамически ограничивает скорость обучения, интерполируя между поведением Adam в начале обучения и SGD ближе к его завершению. Преимуществом является попытка обеспечить как быструю сходимость, так и хорошую обобщающую способность. Недостатком может быть необходимость подбора дополнительных гиперпараметров, связанных с границами скорости обучения.
- **AdaHessian** [1]. Это оптимизатор второго порядка, который аппроксимирует диагональ матрицы Гессе для адаптации скорости обучения. Использование информации о второй производной потенциально позволяет более точно настраивать шаг обновления для каждого параметра. Преимуществом является учет кривизны функции потерь, что может ускорить сходимость на сложных ландшафтах. К недостаткам можно отнести более высокие вычислительные затраты на каждом шаге по сравнению с методами первого порядка, хотя авторы предлагают эффективные способы аппроксимации диагонали Гессеана.
- **AdaBelief** [2]. Модификация Adam, которая адаптирует размер шага в зависимости от «веры» (belief) в текущее направление градиента, измеряемой как разница между наблюдаемым градиентом и экспоненциальным скользящим средним градиентов. Утверждается, что это приводит к лучшей стабильности и производитель-

ности. Преимуществом является интуитивная идея адаптации на основе предсказуемости градиента. Как и другие адаптивные методы, требует настройки гиперпараметров.

Каждый из этих алгоритмов предлагает свой подход к решению проблем, связанных с оптимизацией глубоких нейронных сетей, и выбор наиболее подходящего часто зависит от конкретной задачи, архитектуры модели и доступных вычислительных ресурсов.

2.2. Обзор используемых технологий

Для реализации экспериментов, обучения моделей и анализа результатов был использован следующий набор программных инструментов и библиотек, выбор которых обусловлен их функциональностью и широким распространением в научном сообществе:

- **PyTorch** [13]. В качестве основного фреймворка для глубокого обучения был выбран PyTorch. Это популярный фреймворк с открытым исходным кодом, предоставляющий гибкие инструменты для построения и обучения нейронных сетей через тензорные операции с поддержкой GPU, автоматическое дифференцирование, а также готовые наборы данных, архитектуры моделей и утилиты для обработки изображений (**TorchVision**). Для расчета метрик качества использовалась библиотека **TorchMetrics**, а для получения сводной информации об архитектуре моделей – **TorchInfo**. Выбор PyTorch обоснован его динамическим графом вычислений, что упрощает отладку, обширной поддержкой сообщества, большим количеством готовых реализаций моделей и удобным API, что значительно ускоряет процесс прототипирования и проведения исследований.
- **Ray Tune** [16], являющийся частью экосистемы Ray [14], используется для подбора гиперпараметров. Библиотека предоставляет масштабируемую платформу для распределенного выполнения экспериментов по настройке гиперпараметров, интегрирует

ся с **PyTorch** и позволяет применять различные алгоритмы поиска. Использование **Ray Tune** было продиктовано необходимостью эффективного перебора большого пространства гиперпараметров для каждого оптимизатора и задачи, а также возможностью распараллеливания вычислений для ускорения этого процесса.

- **TensorBoard** [6]. Для мониторинга процесса обучения и визуализации метрик (значений функции потерь, точности и др.) применялся **TensorBoard**. Это стандартный инструмент, позволяющий в реальном времени отслеживать динамику обучения, сравнивать различные запуски и анализировать структуру графа вычислений модели. Его выбор обусловлен простотой интеграции с **PyTorch** и **Ray Tune**, а также наглядностью представления результатов, что критически важно для анализа процесса обучения.
- **Pandas** [10]. Библиотека **Pandas** использовалась для обработки и анализа структурированных данных, в частности, для агрегации результатов экспериментов, полученных из логов обучения и **Ray Tune**, и подготовки их к дальнейшей визуализации. **Pandas** является де-факто стандартом для работы с табличными данными в **Python** благодаря своим мощным и удобным структурам данных, широкому набору функций для манипуляции данными.
- **Seaborn** [18]. Для построения статистических графиков и визуализации сравнительных результатов работы оптимизаторов была выбрана библиотека **Seaborn**. Она является надстройкой над **Matplotlib** и позволяет создавать более информативные диаграммы с меньшим количеством кода. Выбор **Seaborn** обусловлен его способностью легко генерировать сложные типы визуализаций, необходимые для сравнения производительности оптимизаторов.

Реализации исследуемых оптимизаторов были получены из следующих источников:

- **AdamW**: использовалась встроенная реализация из пакета `torch.optim.AdamW` фреймворка **PyTorch**.

- **AdaBelief**: использовалась реализация из сторонней библиотеки `adabelief-pytorch` [19].
- **Lion**: использовалась реализация из сторонней библиотеки `lion-pytorch` [17].
- **AdaBound**: использовалась реализация из сторонней библиотеки `AdaBound` [9].
- **AdaHessian**: использовалась реализация из сторонней библиотеки `torch-optimizer` [11].

Выбор конкретных реализаций был обусловлен их доступностью, активной поддержкой (в случае сторонних библиотек), заявленным соответствием оригинальным алгоритмам, описанным в научных публикациях [8, 15, 3, 1, 2], и простотой интеграции в существующую кодовую базу на PyTorch.

3. Описание решения

В данном разделе описывается программная реализация модулей, необходимых для проведения экспериментов, детали экспериментальной установки, а также подробно рассматривается постановка и результаты решений задач регрессии, классификации и классификации изображений в последующих разделах.

3.1. Реализация программных модулей

Для проведения исследований была разработана структура проекта, включающая следующие ключевые компоненты, расположенные в директории `src/`:

- `config.py`: Файл для инициализации глобальных переменных и констант, таких как пути к данным, параметры обучения по умолчанию, а также для предварительной настройки окружения (установка `seed` для воспроизводимости).
- `data_setup.py`: Модуль, отвечающий за предобработку данных. Включает функции для загрузки исходных данных, их нормализации и создания объектов `Dataset` и `DataLoader` из фреймворка `PyTorch` для последующей подачи в модель.
- `utils.py`: Содержит вспомогательные функции, используемые в различных частях проекта:
 - Функция для создания экземпляра `SummaryWriter` из `TensorBoard` для логирования метрик и параметров обучения.
 - Функции визуализации: `plot_results` для построения графиков изменения функции потерь и выбранных метрик в зависимости от эпохи обучения, что позволяет отслеживать динамику. Функции `plot_confusion_matrices_results`, `plot_roc_results` для построения матриц ошибок и ROC-кривых для задач классификации.

- `optimizers.py`: В данном модуле были внесены незначительные корректировки в существующие реализации некоторых оптимизаторов для обеспечения совместимости с используемыми версиями PyTorch и другими зависимостями.
- `model_builder.py`: Модуль для определения архитектур нейронных сетей. Включает реализацию двух базовых моделей:
 - `SimpleRegressionModel`: Небольшая полносвязная нейронная сеть, предназначенная для решения задач регрессии.
 - `SimpleClassificationModel`: Аналогичная полносвязная сеть, адаптированная для задач бинарной или многоклассовой классификации.
- `engine.py`: Реализует основной цикл обучения и валидации моделей. Содержит функции для выполнения одного шага обучения (`train_step`) и одного шага валидации (`valid_step`), включая вычисление функции потерь, обратное распространение ошибки, шаг оптимизатора и расчет метрик.

Эксперименты и анализ результатов проводились с использованием ноутбуков Jupyter Notebook (`.ipynb`), расположенных в директории `analyzes/`. Исходные наборы данных, используемые в экспериментах, хранятся в директории `data/`.

4. Экспериментальное исследование

В данном разделе представлено описание экспериментальной установки, формулируются исследовательские вопросы, определяются используемые метрики и приводятся результаты сравнительного анализа алгоритмов оптимизации на трех различных задачах: регрессии, классификации и классификации изображений.

4.1. Условия проведения экспериментов

Все вычислительные эксперименты проводились на облачной платформе **Kaggle Notebooks**. Аппаратная конфигурация включала:

- Центральный процессор: 4-ядерный Intel Xeon CPU @ 2.00GHz.
- Графические ускорители: 2 x NVIDIA Tesla T4 GPU. Наличие двух GPU позволяло эффективно распараллеливать запуски при подборе гиперпараметров с использованием библиотеки **Ray Tune** [16].
- Оперативная память: 32GB RAM.

Программное окружение было основано на стандартных контейнерах **Kaggle** и включало следующие ключевые версии библиотек:

- **PyTorch** — версия 2.1.2 (или более поздняя, совместимая).
- **TorchVision** — версия 0.16.2 (или более поздняя, совместимая).
- **Ray (Ray Tune)** — версия 2.9.3 (или более поздняя, совместимая).
- **Pandas** — версия 2.0.3 (или более поздняя, совместимая).
- **Seaborn** — версия 0.13.1 (или более поздняя, совместимая).

Для обеспечения воспроизводимости результатов экспериментов в рамках каждого отдельного запуска (включая подбор гиперпараметров и финальное обучение) использовалась самописная функция **setup**, которая фиксировала начальное состояние генераторов случайных чисел

(seed) для Python, NumPy и PyTorch, а также обеспечивала перезапуск окружения Ray с идентичными параметрами при необходимости.

4.2. Исследовательские вопросы

Основной целью данной работы является сравнение современных алгоритмов оптимизации. В связи с этим были сформулированы следующие исследовательские вопросы (RQ):

RQ1: Какой из исследуемых алгоритмов оптимизации (AdaHessian, Lion, AdaBelief, AdamW, AdaBound) демонстрирует наилучшую производительность (качество модели) на различных типах задач (регрессия, табличная классификация, классификация изображений) при использовании как подобранных, так и стандартных гиперпараметров?

RQ2: Насколько существенна разница во времени обучения между исследуемыми оптимизаторами при достижении сопоставимого качества модели на различных задачах?

RQ3: Является ли подбор гиперпараметров критически важным для достижения высокой производительности для всех исследуемых оптимизаторов, или некоторые из них показывают хорошие результаты с параметрами по умолчанию?

Ответы на эти вопросы позволят сформулировать рекомендации по выбору оптимизаторов для практического применения.

4.3. Метрики оценки

Для оценки качества моделей и сравнения оптимизаторов использовались следующие метрики:

- Для задачи регрессии:
 - *Функция потерь (Loss)*: Среднеквадратичная ошибка (Mean Squared Error, MSE).

- *Средняя абсолютная процентная ошибка (Mean Absolute Percentage Error, MAPE)*: Показывает среднее отклонение прогнозов от фактических значений в процентах, полезна для интерпретации масштаба ошибок.
- *Коэффициент детерминации (R^2 Score)*: Доля дисперсии зависимой переменной, объясняемая рассматриваемой моделью. Значения ближе к 1 указывают на лучшее качество модели.
- **Для задач классификации (включая классификацию изображений):**
 - *Функция потерь (Loss)*: Перекрестная энтропия (Cross-Entropy Loss).
 - *Точность (Accuracy)*: Доля правильно классифицированных объектов.
 - *Точность (Precision, macro-averaged)*: Способность алгоритма не относить объект к некоторому классу, если он к нему не принадлежит. Усредняется по всем классам.
 - *Полнота (Recall, macro-averaged)*: Способность алгоритма обнаруживать объекты данного класса. Усредняется по всем классам.
 - *F1-мера (F1-score, micro-averaged для общей оценки, macro-averaged для покласового анализа при необходимости)*: Среднее гармоническое точности и полноты. F1-score (micro) численно равна Accuracy в задачах многоклассовой классификации.
 - *Площадь под ROC-кривой (AUC ROC, One-vs-Rest)*: Агрегированная метрика качества бинарных/многоклассовых классификаторов, построенных для каждого класса против всех остальных.
- **Общие метрики для всех задач:**

- *Время обучения (Training Time)*: Общее время, затраченное на обучение модели за заданное количество эпох.

Выбор данных метрик обусловлен их широким распространением в задачах соответствующего типа и способностью комплексно оценить различные аспекты производительности моделей.

4.4. Результаты экспериментов

В данном разделе приводятся результаты экспериментов для каждой из трех поставленных задач. Для каждой задачи сначала описывается набор данных и архитектура модели, затем процесс настройки гиперпараметров и, наконец, анализируются полученные результаты.

4.4.1. Эксперимент 1: Задача регрессии

Описание набора данных. Для задачи регрессии использовался набор данных «California Housing Prices» [12]. Этот датасет содержит данные о жилье в Калифорнии, собранные по результатам переписи населения США 1990 года.

- **Размер:** 20640 наблюдений (блоков домов).
- **Признаки (8 числовых):** MedInc (медианный доход в квартале), HouseAge (медианный возраст домов в квартале), AveRooms (среднее количество комнат на домохозяйство), AveBedrms (среднее количество спален на домохозяйство), Population (население квартала), AveOccup (средняя заселенность домохозяйств), Latitude (широта квартала), Longitude (долгота квартала).
- **Целевая переменная:** MedianHouseValue (медианная стоимость дома в квартале), выраженная в долларах.

Перед подачей в модель данные были нормализованы с использованием `StandardScaler`.

Архитектура модели. Для решения задачи регрессии использовалась модель `SimpleRegressionModel`. Ее архитектура, представленная с помощью `TorchInfo`, приведена в таблице 1.

Таблица 1: Архитектура модели SimpleRegressionModel

Layer (type:depth-idx)	Param #
SimpleRegressionModel	--
└ Sequential: 1-1	--
└ Linear: 2-1	576
└ ReLU: 2-2	--
└ Linear: 2-3	4,160
└ ReLU: 2-4	--
└ Linear: 2-5	65
Total params:	4,801

Модель состоит из трех полносвязных слоев с функциями активации ReLU после первых двух. Выходной слой линейный, так как решается задача регрессии.

Настройка гиперпараметров. Обучение проводилось в течение NUM_EPOCHS = 31 эпох с шагом сохранения результатов EPOCH_STEP = 1. Размер батча (BATCH_SIZE) составлял 128. Подбор гиперпараметров для каждого оптимизатора осуществлялся с помощью Ray Tune. Исследуемые значения для каждого параметра представлены ниже:

AdaHessian:

- lr: [1e-3, 1e-2, 1e-1, 0.15, 1.0]
- weight_decay: [0, 1e-4, 1e-3, 1e-2]
- hessian_power: [0.5, 0.75, 1.0]

Lion:

- lr: [1e-4, 1e-3, 1e-2, 1e-1, 0.15]
- weight_decay: [1e-4, 1e-3, 1e-2, 1e-1]
- decoupled_weight_decay: [False, True]
- use_triton: [False]

AdaBelief:

- lr: [1e-4, 1e-3, 1e-2, 1e-1, 0.15]
- weight_decay: [0, 1e-4, 1e-3, 1e-2, 1e-1]
- amsgrad: [False, True]
- weight_decouple: [False, True]
- rectify: [False, True]
- print_change_log: [False]

AdamW:

- lr: [1e-4, 1e-3, 1e-2, 1e-1, 0.15]
- weight_decay: [0, 1e-4, 1e-3, 1e-2, 1e-1]
- amsgrad: [False, True]

AdaBound:

- lr: [1e-7, 1e-6, 1e-5, 1e-4]
- final_lr: [1e-7, 1e-6, 1e-5]
- gamma: [1e-4, 1e-3, 1e-2, 1e-1]
- weight_decay: [0, 1e-4, 1e-3, 1e-2]
- amsbound: [False, True]

Результаты подбора гиперпараметров. В результате работы Ray Tune были определены лучшие конфигурации гиперпараметров для каждого оптимизатора по метрикам MAPE (Mean Absolute Percentage Error) и R^2 Score (коэффициент детерминации) на валидационной выборке. Результаты представлены в таблицах 2 и 3.

Таблица 2: Лучшие конфигурации оптимизаторов по метрике MAPE для задачи регрессии

Optimizer	Config	MAPE	Time_total_s
AdaHessian	{'lr': 0.1, 'wd': 0, 'hp': 1.0, 's': 0}	0.206933	46.053466
Lion	{'lr': 0.1, 'wd': 1e-4, 'dwd': True, 'ut': False}	0.217938	31.553410
AdaBelief	{'lr': 0.1, 'wd': 0.01, 'ag': True, 'wdec': True, 'r': False, 'pcl': False}	0.247575	34.516403
AdamW	{'lr': 0.1, 'wd': 0.01, 'ag': False}	0.247539	31.026601
AdaBound	{'lr': 1e-5, 'flr': 1e-7, 'g': 0.01, 'wd': 1e-3, 'ab': False}	0.207838	35.023025

Сокращения: wd: weight_decay, hp: hessian_power, s: seed, dwd: decoupled_weight_decay, ut: use_triton, ag: amsgrad, wdec: weight_decouple, r: rectify, pcl: print_change_log, flr: final_lr, g: gamma, ab: amsbound.

Таблица 3: Лучшие конфигурации оптимизаторов по метрике R^2 Score для задачи регрессии

Optimizer	Config	R^2 Score	Time_total_s
AdaHessian	{'lr': 0.01, 'wd': 1e-3, 'hp': 0.75, 's': 0}	0.756148	46.411203
Lion	{'lr': 0.1, 'wd': 1e-4, 'dwd': True, 'ut': False}	0.747879	31.553410
AdaBelief	{'lr': 0.1, 'wd': 1e-3, 'ag': False, 'wdec': True, 'r': False, 'pcl': False}	0.706242	33.956667
AdamW	{'lr': 0.1, 'wd': 1e-3, 'ag': False}	0.704664	30.978518
AdaBound	{'lr': 1e-4, 'flr': 1e-7, 'g': 0.01, 'wd': 0, 'ab': False}	0.729952	33.727226

Сокращения: wd: weight_decay, hp: hessian_power, s: seed, dwd: decoupled_weight_decay, ut: use_triton, ag: amsgrad, wdec: weight_decouple, r: rectify, pcl: print_change_log, flr: final_lr, g: gamma, ab: amsbound.

Следует отметить, что конфигурации в таблицах сокращены для читаемости; полные конфигурации соответствуют сетке поиска.

Анализ результатов и сравнение с параметрами по умолчанию.

Был проведен дополнительный анализ сравнения производительности регрессионной модели на тестовой выборке после обучения в течение увеличенного числа эпох (50 эпох). Сравнивались конфигурации гиперпараметров, полученные в результате подбора (используя конфигурации, лучшие по R^2 Score из таблицы 3, как наиболее общую метрику

качества регрессии), с конфигурациями по умолчанию для каждого из оптимизаторов. Оценка проводилась по метрикам тестовой выборки: `test_loss`, `test_mape`, `test_r2score`, а также по общему времени обучения (`time_training`). Результаты этого сравнения сведены в таблицу 4.

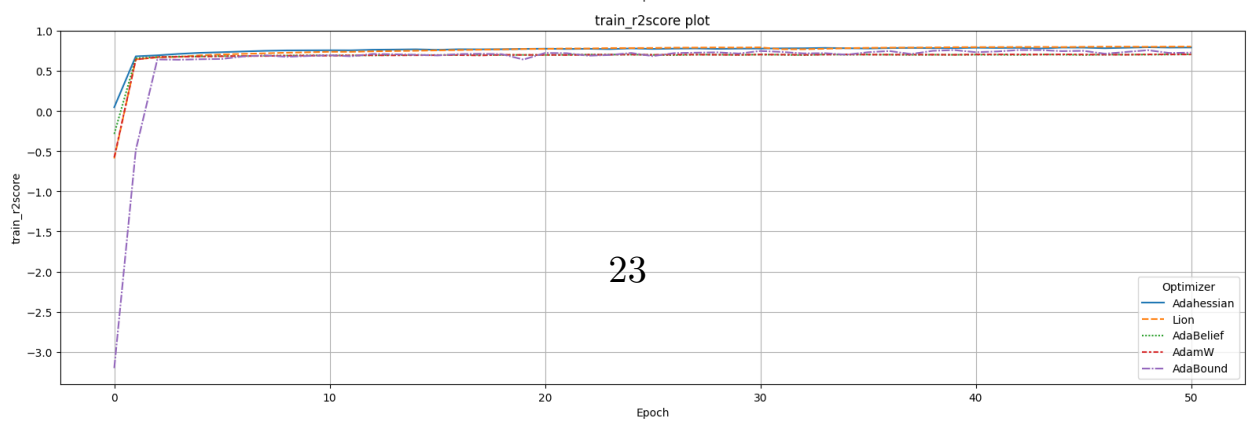
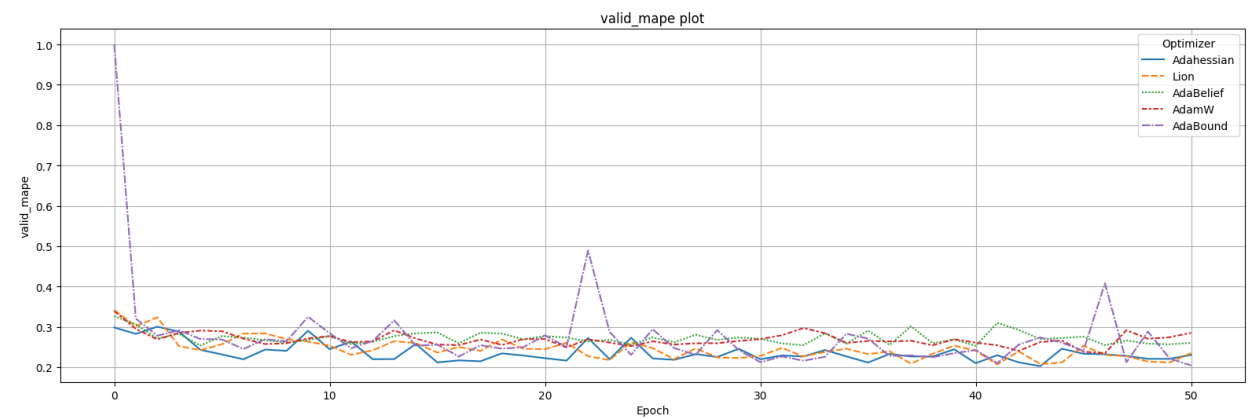
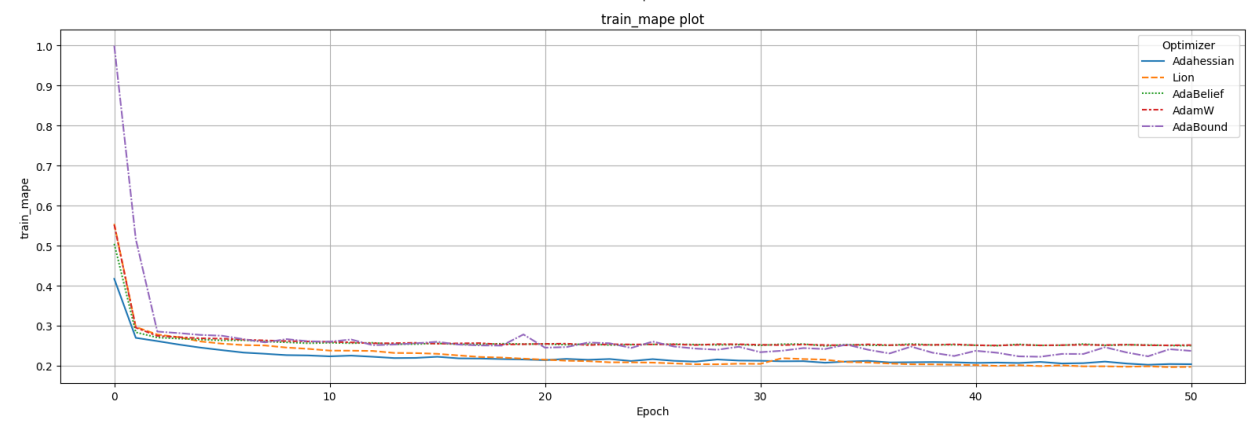
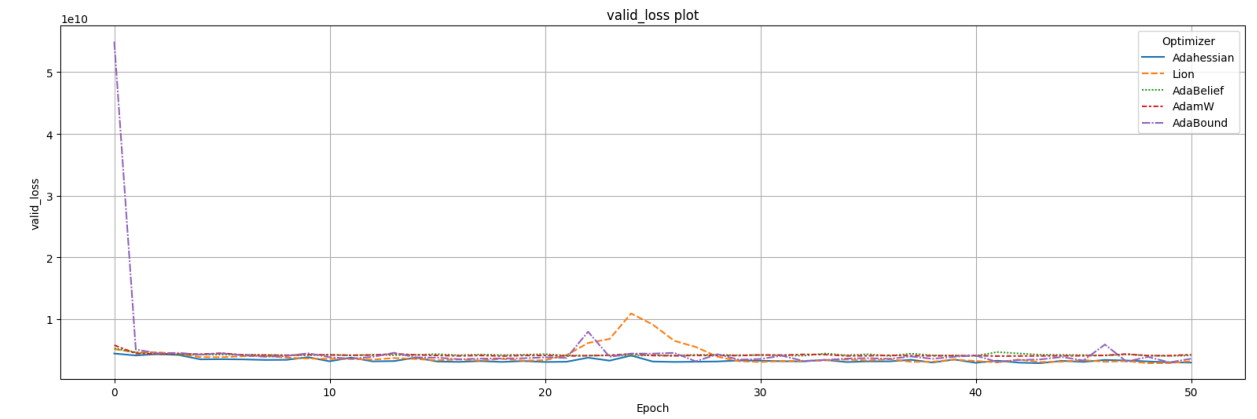
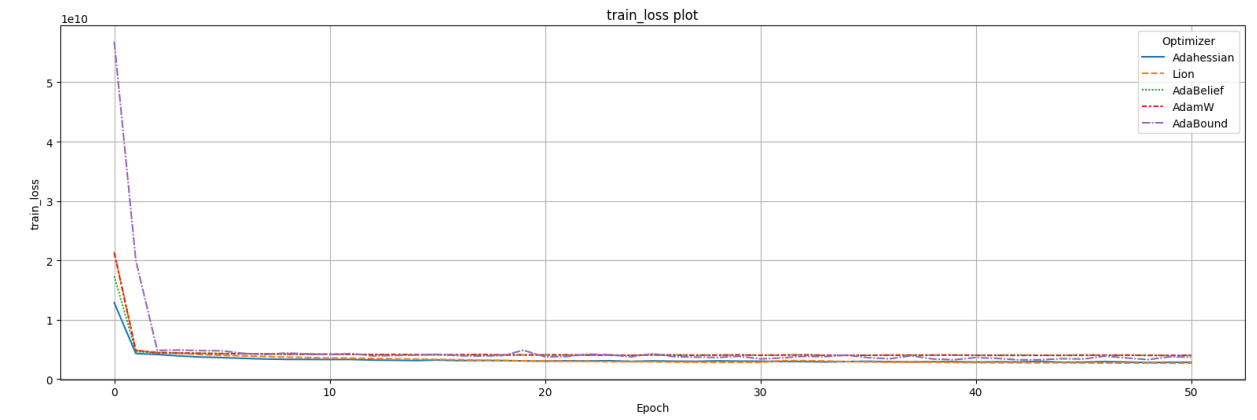
Таблица 4: Сравнение подобранных и стандартных конфигураций оптимизаторов на тестовой выборке для задачи регрессии

Optimizer	test_loss	test_mape	test_r2score	time_training (s)	Config
AdaHessian(tuned)	3.149e+09	0.231462	0.753009	48.78	{'lr': 0.01, 'wd': 0.001, 'hp': 0.75, 's': 0}
Lion(tuned)	3.428e+09	0.239641	0.731702	37.63	{'lr': 0.1, 'wd': 1e-4, 'dwd': True, 'ut': False}
AdaBelief(tuned)	4.243e+09	0.259151	0.668290	38.53	{'lr': 0.1, 'wd': 0.001, 'ag': False, 'wdec': True, 'r': False, 'pcl': False}
AdamW(tuned)	4.360e+09	0.282246	0.659101	36.96	{'lr': 0.1, 'wd': 0.001, 'ag': False}
AdaBound(tuned)	3.894e+09	0.209626	0.697066	38.58	{'lr': 1e-4, 'flr': 1e-7, 'g': 0.01, 'wd': 0, 'ab': False}
AdaHessian(default)	3.243e+09	0.236388	0.746907	47.50	{'lr': 0.15, 'betas': (0.9,0.999), 'eps': 1e-4, 'wd': 0, 'hp': 0.5, 's': None}
Lion(default)	3.287e+09	0.237643	0.743554	37.64	{'lr': 1e-4, 'betas': (0.9,0.99), 'wd': 0.0, 'ut': False, 'dwd': False}
AdaBelief(default)	4.195e+09	0.245528	0.673435	38.37	{'lr': 1e-3, 'betas': (0.9,0.999), 'eps': 1e-16, 'wd': 0, 'ag': False, 'wdec': True, 'fd': False, 'r': True, 'dts': True, 'pcl': True}
AdamW(default)	4.190e+09	0.257028	0.673274	36.25	{'lr': 1e-3, 'betas': (0.9,0.999), 'eps': 1e-08, 'wd': 0.01, 'ag': False}
AdaBound(default)	3.896e+09	0.246281	0.695300	38.59	{'lr': 1e-3, 'betas': (0.9,0.999), 'flr': 0.1, 'g': 1e-3, 'eps': 1e-08, 'wd': 0, 'ab': False}

Сокращения для Config: wd: weight_decay, hp: hessian_power, s: seed, dwd: decoupled_weight_decay, ut: use_triton, ag: amsgrad, wdec: weight_decouple, r: rectify, pcl: print_change_log, flr: final_lr, g: gamma, ab: amsbound, fd: fixed_decay, dts: degenerated_to_sgd. (tuned) - конфигурация после подбора.

Вывод о целесообразности подбора гиперпараметров для задачи регрессии: Анализ данных из таблицы 4 показывает, что подбор гиперпараметров привел к улучшению производительности по некоторым оптимизаторам в сравнении с их конфигурациями по умолчанию. Для `AdaHessian` и `AdaBound` подобранные конфигурации показали лучшие или сопоставимые с лучшими результаты на тестовой выборке по сравнению с по умолчанию, что указывает на потенциальную пользу от подбора для этих оптимизаторов на данной задаче. Например, `AdaBound(tuned)` достиг MAPE 0.209626 против 0.246281 у `AdaBound(default)`. `AdaHessian(tuned)` показал R^2 Score 0.753009 против 0.746907 у `AdaHessian(default)`. Для `Lion`, `AdaBelief` и `AdamW` конфигурации по умолчанию превзошли подобранные по показателям метрик на тестовой выборке. Например, `Lion(default)` показал R^2 Score 0.743554 против 0.731702 у `Lion(tuned)`. Таким образом, на данном наборе данных и с данной моделью, подбор гиперпараметров не гарантировал улучшения производительности на тестовой выборке по сравнению с параметрами по умолчанию для всех исследованных оптимизаторов. Целесообразность подбора оказалась зависимой от конкретного оптимизатора.

Визуализация процесса обучения. На рисунке 1 представлены графики изменения функции потерь (`train_loss`, `valid_loss`), метрики MAPE (`train_mape`, `valid_mape`) и R^2 Score (`train_r2score`, `valid_r2score`) в процессе обучения для лучших (подобранных) конфигураций каждого оптимизатора. Анализируя графики на рисунке 1, можно отметить следующее: `AdaBound` демонстрирует значительно большее значение функции потерь на начальных эпохах, а также более высокую зашумленность кривых обучения на протяжении всего процесса. У оптимизатора `Lion` наблюдается заметный пик значения потерь на валидационной выборке примерно на 25-й эпохе. В целом, все рассмотренные оптимизаторы показывают сопоставимую скорость сходимости к близким значениям метрик после начального этапа обучения, хотя и с некоторыми различиями в стабильности и поведении на валидационной



выборке.

4.4.2. Эксперимент 2: Задача классификации

Описание набора данных. Для задачи классификации использовался набор данных «Covertypes» (Forest Cover Type) из репозитория UCI Machine Learning Repository [5, ?]. Этот датасет содержит картографические переменные, предсказывающие тип лесного покрова для участков размером 30x30 метров в национальном лесу Рузвельт в северной части штата Колорадо.

- **Полный размер:** 581012 наблюдений.
- **Используемая часть:** В связи с большим объемом данных и для обеспечения разумного времени обучения, в экспериментах использовалась случайная выборка, составляющая 10% от полного набора данных (58101 наблюдение).
- **Признаки:** 54 признака, включающие 10 количественных (например, высота над уровнем моря, уклон, расстояние до водных объектов) и 44 бинарных (4 признака типа дикой местности и 40 признаков типа почвы).
- **Целевая переменная:** Тип лесного покрова (Cover_Type), представленный 7 классами (например, ель/пихта, сосна Лоджполя, сосна пондероза и т.д.).

Количественные признаки были нормализованы с использованием `StandardScaler`.

Архитектура модели. Для решения задачи классификации использовалась модель `SimpleClassificationModel`. Ее архитектура, представленная с помощью `TorchInfo`, приведена в таблице 5. Модель состоит из двух полносвязных слоев. Выходной слой имеет 7 нейронов (по числу классов) и не включает явную функцию активации `Softmax`, так как она обычно комбинируется с функцией потерь `CrossEntropyLoss` в `PyTorch`.

Таблица 5: Архитектура модели SimpleClassificationModel

Layer (type:depth-idx)	Param #
SimpleClassificationModel	--
└ Sequential: 1-1	--
└ Linear: 2-1	2,970
└ ReLU: 2-2	--
└ Linear: 2-3	385
Total params:	3,355

Настройка гиперпараметров. Обучение проводилось в течение `NUM_EPOCHS = 26` эпох с шагом сохранения результатов `EPOCH_STEP = 1`. Размер батча (`BATCH_SIZE`) составлял 256. Подбор гиперпараметров для каждого оптимизатора осуществлялся с помощью `Ray Tune`. Исследуемые значения для каждого параметра представлены ниже:

AdaHessian:

- `lr`: [1e-2, 1e-1, 0.15, 1.0]
- `weight_decay`: [0, 1e-4, 1e-3, 1e-2]
- `hessian_power`: [0.5, 0.75, 1.0]

Lion:

- `lr`: [1e-4, 1e-3, 1e-2, 1e-1]
- `weight_decay`: [1e-4, 1e-3, 1e-2]
- `decoupled_weight_decay`: [False, True]
- `use_triton`: [False]

AdaBelief:

- `lr`: [1e-4, 1e-3, 1e-2, 1e-1]
- `weight_decay`: [0, 1e-4, 1e-3, 1e-2]
- `amsgrad`: [False, True]

- `weight_decouple`: [False, True]
- `rectify`: [False, True]
- `print_change_log`: [False]

AdamW:

- `lr`: [1e-4, 1e-3, 1e-2, 1e-1]
- `weight_decay`: [0, 1e-4, 1e-3, 1e-2]
- `amsgrad`: [False, True]

AdaBound:

- `lr`: [1e-4, 1e-3, 1e-2]
- `final_lr`: [1e-3, 1e-2, 1e-1]
- `gamma`: [1e-4, 1e-3, 1e-2]
- `weight_decay`: [0, 1e-4, 1e-3]
- `amsbound`: [False, True]

Результаты подбора гиперпараметров. В результате работы Ray Tune были определены лучшие конфигурации гиперпараметров для каждого оптимизатора по метрике Ассигасу на валидационной выборке. Результаты представлены в таблице 6.

Таблица 6: Лучшие конфигурации оптимизаторов по метрике Accuracy для задачи классификации (Coverttype)

Optimizer	Config	Accuracy	Time_total_s
AdaHessian	{'lr': 0.15, 'wd': 0, 'hp': 0.5, 's': 0}	0.791774	56.416034
Lion	{'lr': 0.001, 'wd': 0.001, 'dwd': False, 'ut': False}	0.797109	43.788289
AdaBelief	{'lr': 0.01, 'wd': 0.001, 'ag': True, 'wdec': True, 'r': False, 'pcl': False}	0.787644	46.663862
AdamW	{'lr': 0.01, 'wd': 0.0001, 'ag': True}	0.786612	45.097362
AdaBound	{'lr': 0.01, 'flr': 0.1, 'g': 0.0001, 'wd': 0.0001, 'ab': False}	0.789881	47.705300

Сокращения: wd: weight_decay, hp: hessian_power, s: seed, dwd: decoupled_weight_decay, ut: use_triton, ag: amsgrad, wdec: weight_decouple, r: rectify, pcl: print_change_log, flr: final_lr, g: gamma, ab: amsbound.

Анализ результатов подбора показывает:

- **Lion** продемонстрировал наивысшую точность (0.7971) и наименьшее время обучения (43.8 с).
- **AdaHessian** показал вторую по величине точность (0.7918), но с наибольшим временем обучения (56.4 с).
- **AdamW** показал наименьшую точность (0.7866), но также одно из наименьших времен обучения (45.1 с).
- Разница в точности между лучшим (Lion) и худшим (AdamW) оптимизатором составила около 1.05%.

Анализ результатов и сравнение с параметрами по умолчанию (50 эпох). Для более глубокого анализа было проведено обучение моделей в течение 50 эпох с использованием как подобранных конфигураций (из таблицы 6), так и конфигураций по умолчанию для каждого оптимизатора. Результаты на тестовой выборке сведены в таблицу 7.

Таблица 7: Сравнение подобранных и стандартных конфигураций оптимизаторов на тестовой выборке (50 эпох) для задачи классификации (Coverttype)

Optimizers	Loss	Acc.	Prec.	Rec.	F1	Time(s)	Config
AdaHessian(t)	0.4857	0.8014	0.7735	0.6357	0.8014	84.65	{'lr':0.15,'wd':0, 'hp':0.5,'s':0}
Lion(t)	0.4842	0.8031	0.7376	0.6646	0.8031	71.43	{'lr':0.001,'wd':0.001, 'dwd':F,'ut':F}
AdaBelief(t)	0.4748	0.8073	0.7411	0.6745	0.8073	72.76	{'lr':0.01,'wd':0.001,'ag':T, 'wdec':T,'r':F,'pcl':F}
AdamW(t)	0.4829	0.7981	0.7406	0.6815	0.7981	71.48	{'lr':0.01,'wd':1e-4, 'ag':T}
AdaBound(t)	0.4937	0.7945	0.7115	0.6476	0.7945	74.97	{'lr':0.01,'flr':0.1,'g':1e-4, 'wd':1e-4,'ab':F}
AdaHessian(d)	0.4870	0.7990	0.7327	0.6725	0.7990	86.50	{'lr':0.15,'b':(0.9,0.999), 'eps':1e-4,'wd':0,'hp':0.5,'s':N}
Lion(d)	0.5466	0.7680	0.6604	0.5591	0.7680	73.32	{'lr':1e-4,'b':(0.9,0.99), 'wd':0,'ut':F,'dwd':F}
AdaBelief(d)	0.5254	0.7754	0.7429	0.5922	0.7754	75.78	{'lr':1e-3,'b':(0.9,0.999), 'eps':1e-16,'wd':0,'ag':F, 'wdec':T,'fd':F,'r':T, 'dts':T,'pcl':T}
AdamW(d)	0.5158	0.7806	0.7014	0.5841	0.7806	73.59	{'lr':1e-3,'b':(0.9,0.999), 'eps':1e-8,'wd':0.01,'ag':F}
AdaBound(d)	0.5270	0.7785	0.6890	0.5911	0.7785	75.21	{'lr':1e-3,'b':(0.9,0.999), 'flr':0.1,'g':1e-3,'eps':1e-8, 'wd':0,'ab':F}

Сокращения для Config: wd: weight_decay, hp: hessian_power, s: seed, dwd: decoupled_weight_decay (F-False, T-True), ut: use_triton (F-False), ag: amsgrad (F-False, T-True), wdec: weight_decouple (F-False, T-True), r: rectify (F-False, T-True), pcl: print_change_log (F-False, T-True), flr: final_lr, g: gamma, ab: amsbound (F-False, T-True), fd: fixed_decay (F-False, T-True), dts: degenerated_to_sgd (F-False, T-True), b: betas, N: None. (t) - tuned, (d) - default. Метрики: Acc. - accuracy, Prec. - precision (macro), Rec. - recall (macro), F1 - F1-score (micro).

Анализ результатов (50 эпох): Сравнение подобранных и стандартных конфигураций: Для задачи классификации на датасете Coverttype подбор гиперпараметров привел к улучшению качества моделей на тестовых данных для всех рассмотренных оптимизаторов. Наибольшее улучшение наблюдалось для оптимизаторов **Lion** (Accuracy/F1 +3.5%), **AdaBelief** (Accuracy/F1 +3.2%), **AdamW** (Accuracy/F1 +1.75%) и **AdaBound** (Accuracy/F1 +1.6%), чьи стандартные настройки показали менее высокие результаты. Для **AdaHessian** улучшение было незначительным.

Выбор лучшего оптимизатора (на основе тестовых данных, 50 эпох):

- *Метрики качества:* **AdaBelief(tuned)** показывает наивысшие значения Accuracy/F1 (0.8073) и наименьшее значение Loss (0.4748). За ним следует **Lion(tuned)** с Accuracy/F1 0.8031.
- *Время выполнения:* Наименьшее время выполнения показывают **Lion(tuned)** (71.43 с) и **AdamW(tuned)** (71.48 с). **AdaBelief(tuned)** (72.76 с) имеет близкое к ним время. **AdaHessian** характеризовался наибольшим временем обучения (84-86 с).
- *Сбалансированный выбор:* **AdaBelief(tuned)** демонстрирует сочетание высоких показателей качества и времени выполнения, близкого к наименьшим значениям. **Lion(tuned)** может рассматриваться при приоритете наименьшего времени выполнения и допустимости снижения точности на 0.4% по сравнению с **AdaBelief(tuned)**.

Итоговые выводы по задаче классификации:

1. **Лучший оптимизатор:** На основе тестовых данных для задачи классификации Coverttype, **AdaBelief**с подобранными параметрами ('lr': 0.01, 'weight_decay': 0.001, 'amsgrad': True, ...) показывает наивысшие значения точности и **F1-score** при времени обучения, близком к минимальному. **Lion** (с подобранными параметрами) является альтернативой с наименьшим временем обучения и незначительно меньшей точностью.

2. Целесообразность подбора гиперпараметров: Для данной задачи подбор гиперпараметров привел к существенному улучшению качества моделей на тестовых данных по сравнению с использованием параметров по умолчанию для большинства оптимизаторов. Это указывает на значимость настройки гиперпараметров для достижения более высоких показателей производительности.

Визуализация процесса обучения. На рисунке 2 представлены графики изменения функции потерь и ключевых метрик в процессе обучения для лучших (подобранных) конфигураций каждого оптимизатора. На рисунке 3 приведены матрицы ошибок, а на рисунке 4 – ROC-кривые.

Анализ графиков (рис. 2, 3, 4): Оптимизатор `Lion` начинает с относительно высоких значений функции потерь на первых эпохах, но затем демонстрирует быструю сходимость; кривая обучения для него выглядит гладкой. На графике `valid_loss` наблюдаются заметные колебания для `AdamW`; для остальных оптимизаторов эти колебания выражены в меньшей степени. На графиках метрик `train_f1score` и `valid_f1score` кривые для `AdamW` и `AdaBound` остаются более зашумленными по сравнению с другими. `Lion` снова показывает хорошую и стабильную сходимость. Для метрики `valid_recall` оптимизатор `AdaHessian` демонстрирует плохую сходимость и сильную зашумленность, особенно на первых 10-15 эпохах, где отсутствует существенный прирост. Вид кривых для остальных оптимизаторов по этой метрике примерно совпадает. Анализ матриц ошибок (рис. 3) показывает, что малопредставленные классы (например, 4, 5, 6) относительно точно, по сравнению с другими оптимизаторами, классифицируются моделью, обученной с использованием `AdamW` (для 5-6 классов) и `AdaBelief` (для 4 класса). ROC-кривые (рис. 4) имеют схожий вид для всех оптимизаторов. `Lion` показывает наибольшее среднее значение AUC.

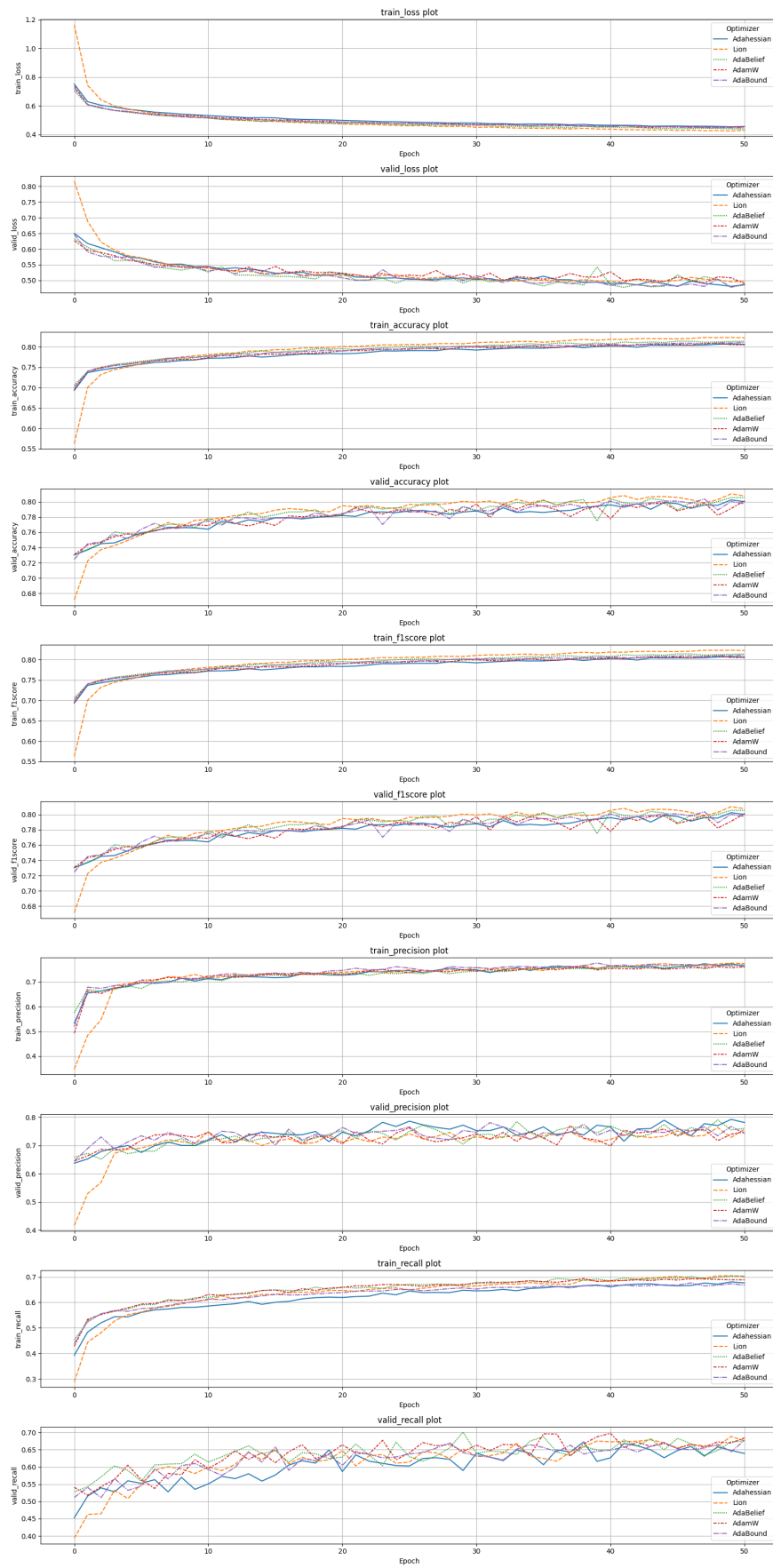


Рис. 2: Динамика функции потерь и метрик (Accuracy, F1, Recall) для задачи классификации (Covertypes).

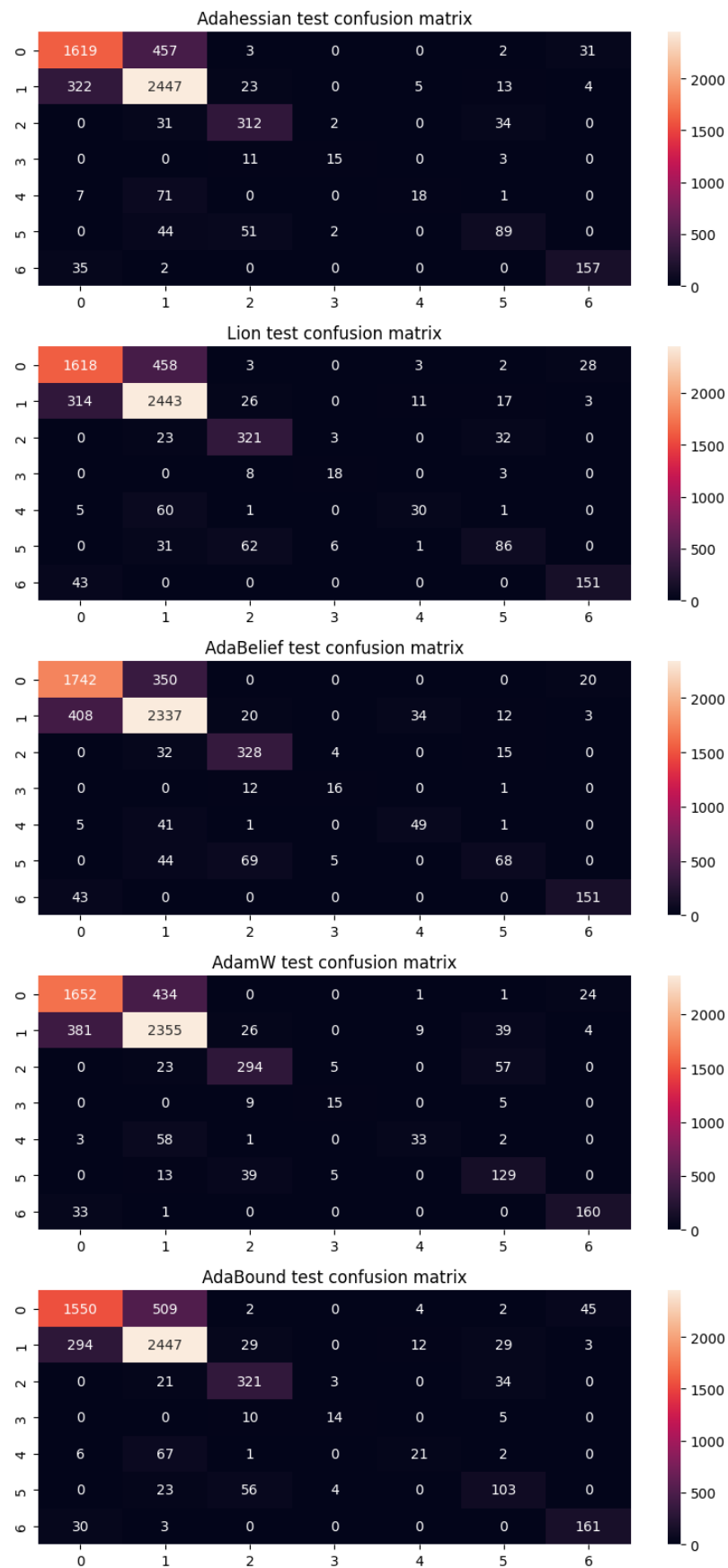


Рис. 3: Матрицы ошибок для различных оптимизаторов на задаче классификации (Coverttype).

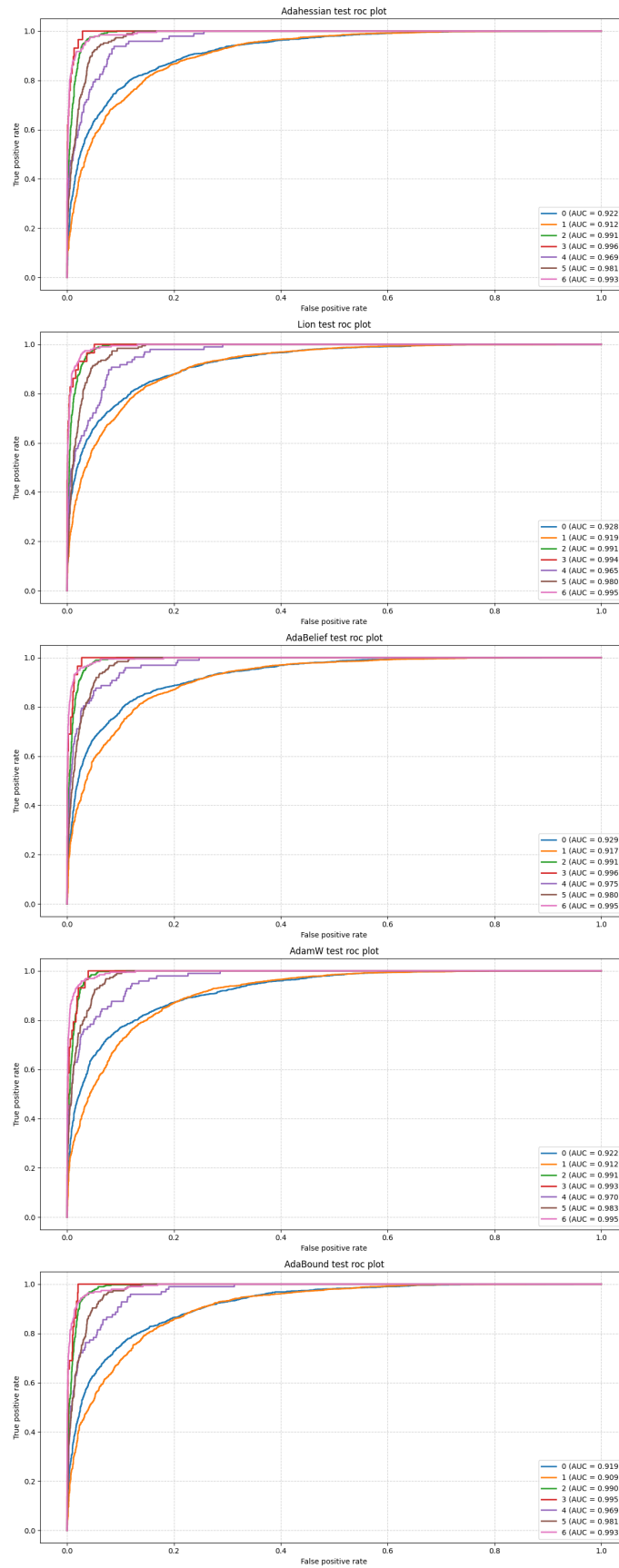


Рис. 4: ROC-кривые (OvR) для различных оптимизаторов на задаче классификации (Coverttype).

4.4.3. Эксперимент 3: Задача классификации изображений

Описание набора данных. Для задачи классификации изображений использовался набор данных CIFAR-10 [7]. Этот датасет является стандартным бенчмарком для оценки алгоритмов машинного обучения и компьютерного зрения.

- **Состав:** 60000 цветных изображений размером 32x32 пикселя, разделенных на 10 классов.
- **Разделение:** 50000 изображений для обучения и 10000 изображений для тестирования. В каждом классе по 6000 изображений.
- **Классы (10):** самолет, автомобиль, птица, кот, олень, собака, лягушка, лошадь, корабль, грузовик.

Изображения были предварительно нормализованы (приведены к диапазону $[0, 1]$ и стандартизированы по среднему и стандартному отклонению для каждого цветового канала на основе обучающей выборки). Применялись также стандартные аугментации данных для обучающей выборки, такие как случайные горизонтальные отражения и случайные вырезания (`RandomResizedCrop` или `RandomCrop` с паддингом).

Архитектура модели. Для решения задачи классификации изображений использовалась предварительно обученная на ImageNet модель ResNet-18 [4], адаптированная для датасета CIFAR-10. ResNet (Residual Network) — это архитектура глубокой сверточной нейронной сети, которая использует остаточные блоки (residual blocks) для облегчения обучения очень глубоких сетей. Остаточные блоки позволяют градиентам легче распространяться через множество слоев, решая проблему затухания градиента. Модель ResNet-18 состоит из 18 слоев, включая сверточные слои, слои пакетной нормализации, функции активации ReLU и один полносвязный слой на выходе. Для CIFAR-10 последний полносвязный слой был заменен на новый, соответствующий 10 классам датасета. Использовалась реализация из `torchvision.models`.

Настройка гиперпараметров. Начальный подбор гиперпараметров проводился в течение `NUM_EPOCHS = 5` эпох с шагом сохранения результатов `EPOCH_STEP = 1`. Размер батча (`BATCH_SIZE`) составлял 256.

Исследуемые значения для каждого параметра при подборе с помощью Ray Tune представлены ниже:

AdaHessian:

- `lr`: [1e-2, 1e-1, 0.15, 1.0]
- `weight_decay`: [0, 1e-4, 1e-3, 1e-2]
- `hessian_power`: [0.5, 0.75, 1.0]

Lion:

- `lr`: [1e-4, 1e-3, 1e-2, 1e-1]
- `weight_decay`: [1e-4, 1e-3, 1e-2]
- `decoupled_weight_decay`: [False, True]
- `use_triton`: [False]

AdaBelief:

- `lr`: [1e-4, 1e-3, 1e-2, 1e-1]
- `weight_decay`: [0, 1e-4, 1e-3, 1e-2]
- `amsgrad`: [False, True]
- `weight_decouple`: [False, True]
- `rectify`: [False, True]
- `print_change_log`: [False]

AdamW:

- `lr`: [1e-4, 1e-3, 1e-2, 1e-1]

- `weight_decay`: [0, 1e-4, 1e-3, 1e-2]
- `amsgrad`: [False, True]

AdaBound:

- `lr`: [1e-5, 1e-4, 1e-3, 1e-2]
- `final_lr`: [1e-4, 1e-3, 1e-2, 1e-1]
- `gamma`: [1e-4, 1e-3, 1e-2]
- `weight_decay`: [0, 1e-4, 1e-3]
- `amsbound`: [False, True]

Результаты подбора гиперпараметров. В результате работы Ray Tune были определены лучшие конфигурации гиперпараметров для каждого оптимизатора по метрике Аккуратности на валидационной выборке после 5 эпох обучения. Результаты представлены в таблице 8.

Таблица 8: Лучшие конфигурации оптимизаторов по метрике Аккуратности для задачи классификации изображений (CIFAR-10, ResNet-18, 5 эпох)

Optimizer	Config	Accuracy	Time_total_s
AdaHessian	{'lr': 0.15, 'wd': 0.0001, 'hp': 0.75, 's': 0}	0.6923	170.386461
Lion	{'lr': 0.001, 'wd': 0.001, 'dwd': False, 'ut': False}	0.7223	69.279341
AdaBelief	{'lr': 0.01, 'wd': 0.001, 'ag': False, 'wdec': True, 'r': True, 'pcl': False}	0.7117	72.120960
AdamW	{'lr': 0.001, 'wd': 0, 'ag': False}	0.6924	68.422461
AdaBound	{'lr': 0.001, 'flr': 0.1, 'g': 0.0001, 'wd': 0.0001, 'ab': False}	0.6988	73.979855

Сокращения: wd: weight_decay, hp: hessian_power, s: seed, dwd: decoupled_weight_decay (F-False, T-True), ut: use_triton (F-False), ag: amsgrad (F-False, T-True), wdec: weight_decouple (F-False, T-True), r: rectify (F-False, T-True), pcl: print_change_log (F-False, T-True), flr: final_lr, g: gamma, ab: amsbound (F-False, T-True).

Анализ результатов подбора показывает:

- **Lion** продемонстрировал наилучшую точность (0.7223) при высокой скорости обучения (69.3 с).
- **AdaBelief** занял второе место по точности (0.7117) со скоростью 72.1 с.
- **AdamW** показал одну из самых низких точностей (0.6924), но оказался самым быстрым (68.4 с).
- **AdaHessian** показал низкую точность (0.6923) и самое длительное время обучения (170.4 с), что более чем в 2.4 раза дольше других.
- Для большинства оптимизаторов оптимальным оказался низкий LR (0.001), за исключением **AdaBelief**(0.01) и **AdaHessian**(0.15).

Анализ результатов и сравнение с параметрами по умолчанию (20 эпох). Для более глубокого анализа было проведено обучение моделей ResNet-18 на CIFAR-10 в течение 20 эпох с использованием как подобранных конфигураций (из таблицы 8), так и конфигураций по умолчанию для каждого оптимизатора. Результаты на тестовой выборке сведены в таблицу 9.

Таблица 9: Сравнение подобранных и стандартных конфигураций оптимизаторов на тестовой выборке (CIFAR-10, ResNet-18, 20 эпох)

Opt.	Loss	Acc.	Prec.	Rec.	F1	Time(s)	Config
AdaHessian(t)	1.4249	0.7283	0.7330	0.7283	0.7283	684.62	{'lr':0.15,'wd':1e-4,'hp':0.75,'s':0}
Lion(t)	1.5380	0.7324	0.7353	0.7324	0.7324	239.82	{'lr':0.001,'wd':0.001,'dwd':F,'ut':F}
AdaBelief(t)	1.1390	0.7531	0.7560	0.7531	0.7531	245.20	{'lr':0.01,'wd':0.001,'ag':F,'wdec':T,'r':T,'pcl':F}
AdamW(t)	1.4335	0.7236	0.7239	0.7236	0.7236	238.73	{'lr':0.001,'wd':0,'ag':F}
AdaBound(t)	1.4658	0.7377	0.7404	0.7377	0.7377	246.82	{'lr':0.001,'flr':0.1,'g':1e-4,'wd':1e-4,'ab':F}
AdaHessian(d)	1.5996	0.7198	0.7271	0.7198	0.7198	689.65	{'lr':0.15,'b':(0.9,0.999),'eps':1e-4,'wd':0,'hp':0.5,'s':N}
Lion(d)	1.6078	0.7073	0.7078	0.7073	0.7073	238.96	{'lr':1e-4,'b':(0.9,0.99),'wd':0,'ut':F,'dwd':F}
AdaBelief(d)	1.5076	0.6896	0.6879	0.6896	0.6896	245.83	{'lr':1e-3,'b':(0.9,0.999),'eps':1e-16,'wd':0,'ag':F,'wdec':T,'fd':F,'r':T,'dts':T,'pcl':T}
AdamW(d)	1.5256	0.7129	0.7266	0.7129	0.7129	238.72	{'lr':1e-3,'b':(0.9,0.999),'eps':1e-8,'wd':0.01,'ag':F}
AdaBound(d)	1.7798	0.7190	0.7205	0.7190	0.7190	243.85	{'lr':1e-3,'b':(0.9,0.999),'flr':0.1,'g':1e-3,'eps':1e-8,'wd':0,'ab':F}

Сокращения для Config: wd: weight_decay, hp: hessian_power, s: seed, dwd: decoupled_weight_decay (F-False, T-True), ut: use_triton (F-False), ag: amsgrad (F-False, T-True), wdec: weight_decouple (F-False, T-True), r: rectify (F-False, T-True), pcl: print_change_log (F-False, T-True), flr: final_lr, g: gamma, ab: amsbound (F-False, T-True), fd: fixed_decay (F-False, T-True), dts: degenerated_to_sgd (F-False, T-True), b: betas, N: None. (t) - tuned, (d) - default. Метрики: Acc. - accuracy, Prec. - precision (macro), Rec. - recall (macro), F1 - F1-score (micro).

Анализ результатов (20 эпох):

• Сравнение подобранных и стандартных конфигураций:

Подбор гиперпараметров оказался важным для достижения хороших результатов на CIFAR-10 с ResNet-18. Улучшения с подобранными параметрами были следующими: **AdaHessian** (Accurasy +0.85%), **Lion** (Accurasy +2.5%), **AdaBelief** (Accurasy +6.35%), **AdamW** (Accurasy +1.1%), **AdaBound** (Accurasy +1.87%). Настройки по умолчанию, особенно для **AdaBelief** и **Lion**, были неоптимальными и привели к более низкому качеству на тестовой выборке после 20 эпох обучения.

• Выбор лучшего оптимизатора (на основе тестовых данных, 20 эпох):

Следует отметить, что при обучении в течение 20

эпох наблюдались признаки переобучения для большинства конфигураций, начиная примерно с 5-10 эпохи (что будет более детально рассмотрено при анализе графиков). Тем не менее, сравнение итоговых метрик на 20-й эпохе позволяет оценить поведение оптимизаторов в таких условиях.

- *Метрики качества:* **AdaBelief(tuned)** показал наилучшие результаты по итоговым метрикам на 20-й эпохе, достигнув Accuracy/F1 0.7531 и наименьшего Loss 1.1390. Ближайший по Accuracy – **AdaBound(tuned)** с 0.7377 (разница 1.5%). Это свидетельствует о том, что даже в условиях начинающегося переобучения, данная конфигурация **AdaBelief** смогла достичь более высокого пика производительности или медленнее деградировала.
- *Время выполнения:* **AdamW(tuned)** (238.7 с) и **Lion(tuned)** (239.8 с) показали наименьшее время обучения за 20 эпох. **AdaBelief(tuned)** (245.2 с) и **AdaBound(tuned)** (246.8 с) были ненамного медленнее. **AdaHessian** (684.6 с) снова оказался значительно медленнее.
- *Сбалансированный выбор с учетом переобучения:* **AdaBelief(tuned)** представляет собой наиболее сбалансированный выбор по итоговым метрикам на 20-й эпохе. Несмотря на то, что его время обучения несколько выше, чем у **AdamW**, он обеспечивает лучшее качество модели (разница по Accuracy составляет около 3%). Однако, для практического применения следовало бы использовать механизмы ранней остановки, ориентируясь на оптимальную точку до начала явного переобучения (около 5-10 эпох, как видно на графиках).

Итоговые выводы по задаче классификации изображений:

1. **Лучший оптимизатор с учетом динамики обучения:**
Для задачи классификации CIFAR-10 с использованием ResNet-

18, **AdaBelief** с подобранными параметрами (`'lr': 0.01, 'weight_decay': 0.001, 'rectify': True, ...`) показал наилучшую общую производительность на 20-й эпохе. Он обеспечивает самую высокую точность, F1-меру и наименьшие потери. Однако, анализ кривых обучения показывает, что оптимальная точка валидации для большинства оптимизаторов достигается раньше (около 5-10 эпох), после чего начинается переобучение. В этом контексте, **Lion(tuned)** также показал очень хорошие результаты на ранних этапах (см. таблицу 8) и мог бы быть предпочтительнее при использовании ранней остановки.

2. **Целесообразность подбора гиперпараметров:** Подбор гиперпараметров был критически важен для этой задачи. Он позволил значительно повысить качество моделей, обученных всеми оптимизаторами, и особенно повлиял на результаты **AdaBelief**, который с настройками по умолчанию показал более низкий результат, а с подобранными — более высокий, даже при более длительном обучении.
3. **Адаптивность AdaHessian:** Из-за значительно большего времени обучения (почти в 3 раза дольше других) **AdaHessian** не является практичным выбором для обучения моделей типа ResNet-18 на CIFAR-10 в данном экспериментальном сеттинге, несмотря на приемлемую точность после подбора параметров.

Визуализация процесса обучения (20 эпох). На рисунке 5 представлены графики изменения функции потерь и ключевых метрик в процессе обучения (20 эпох) для лучших (подобранных) конфигураций каждого оптимизатора. На рисунке 6 приведены матрицы ошибок, а на рисунке 7 – ROC-кривые.

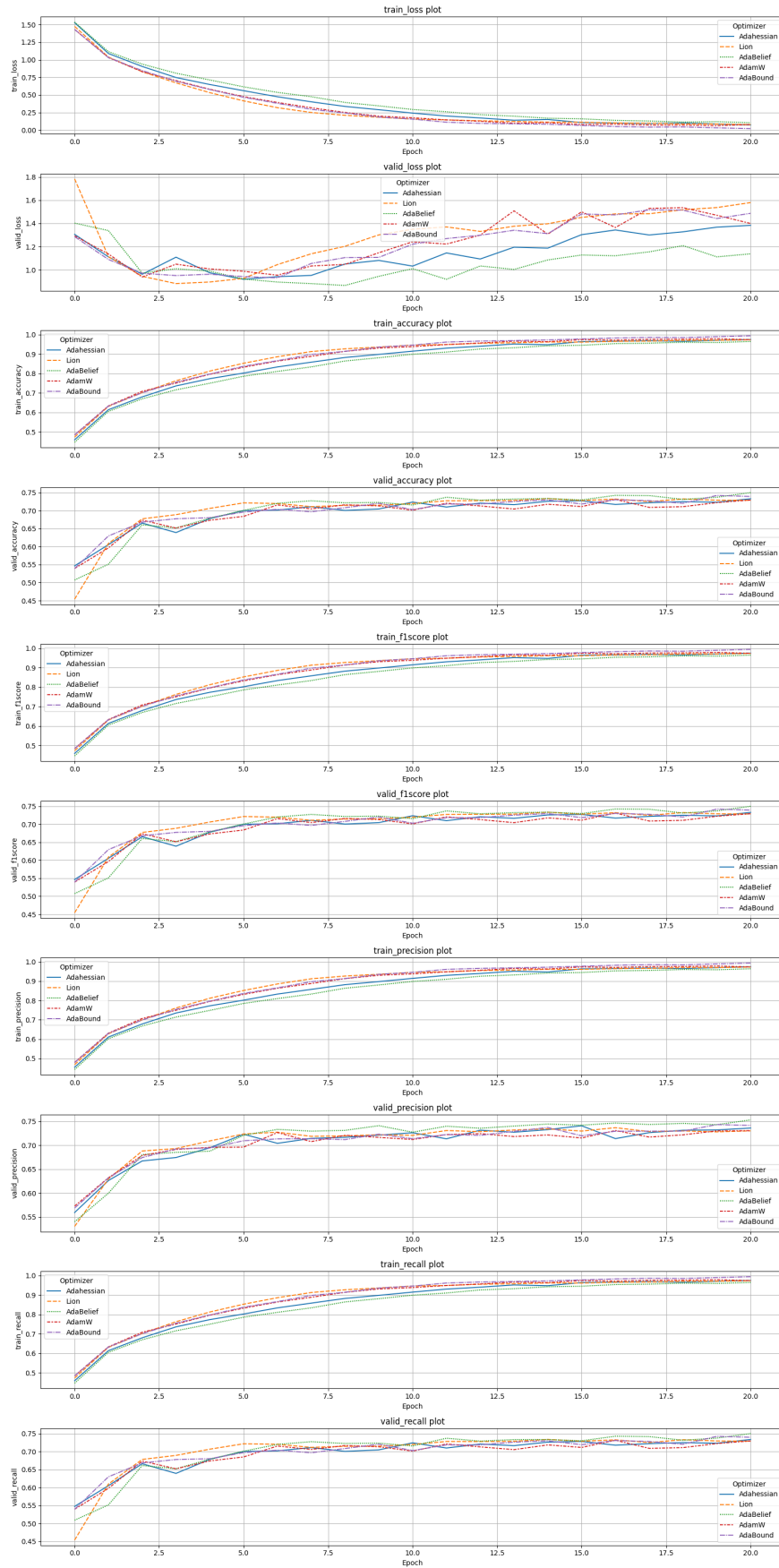


Рис. 5: Динамика функции потерь и метрик для задачи классификации изображений (CIFAR-10, ResNet-18, 20 эпох).

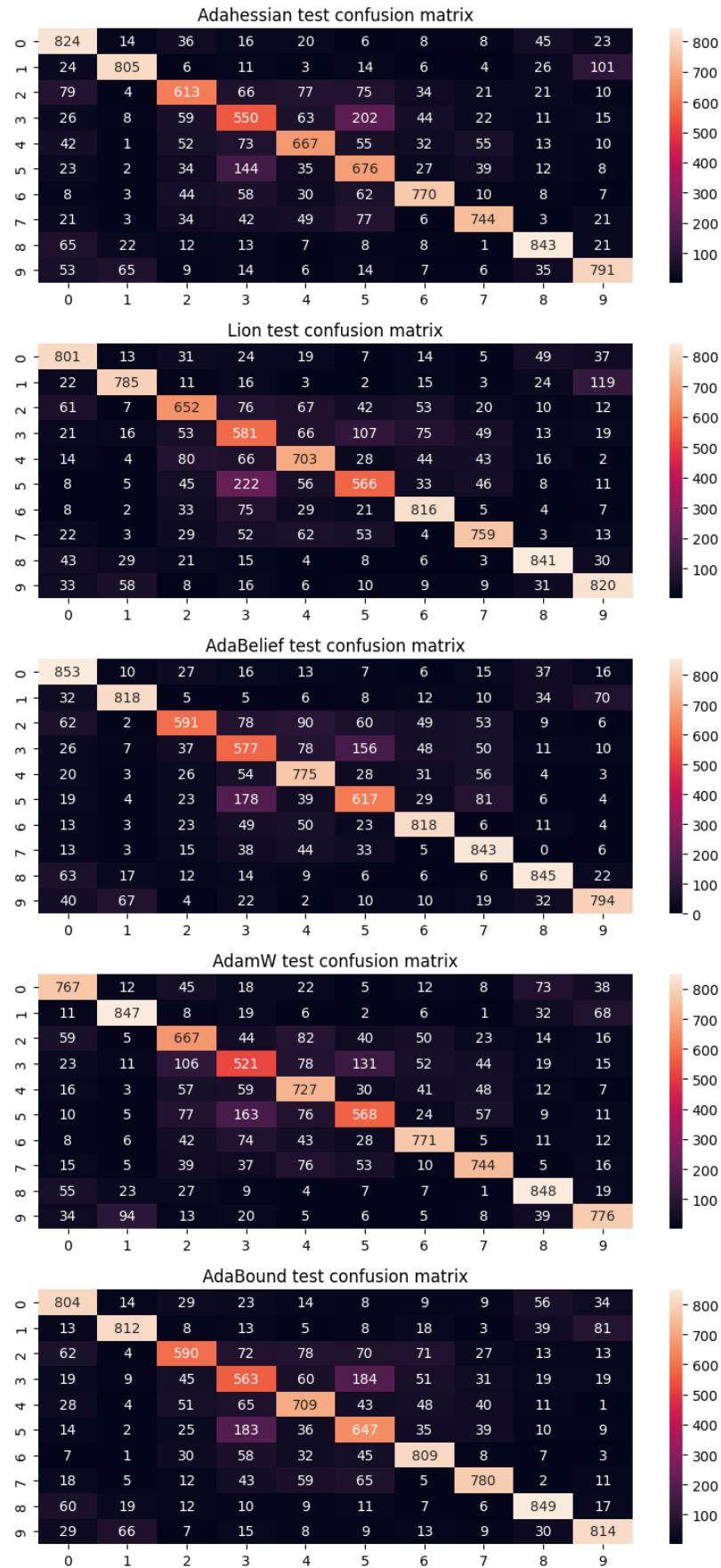


Рис. 6: Матрицы ошибок для различных оптимизаторов на задаче классификации изображений (CIFAR-10, ResNet-18, 20 эпох).

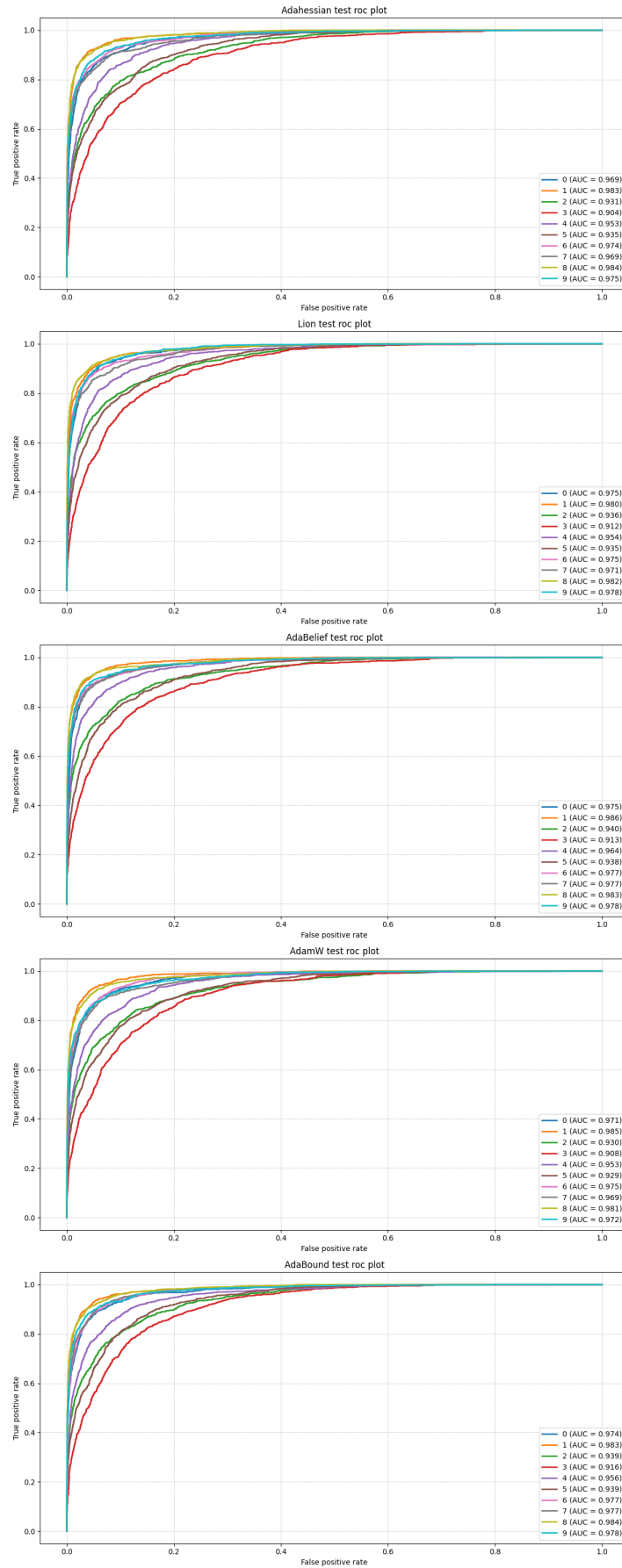


Рис. 7: ROC-кривые (OvR) для различных оптимизаторов на задаче классификации изображений (CIFAR-10, ResNet-18, 20 эпох).

Анализ графиков (рис. 5, 6, 7): На графике `train_loss` (рис. 5) кривые для большинства оптимизаторов имеют схожий вид. `Lion` показывает более быструю начальную сходимость, в то время как `AdaBelief` демонстрирует более медленную, но стабильную сходимость к низким значениям потерь. На графике `valid_loss` видно, что для многих оптимизаторов модели достигают минимального значения потерь примерно на 5-й эпохе, после чего потери на валидационной выборке начинают возрастать. Это свидетельствует о переобучении моделей при дальнейшем обучении до 20 эпох без дополнительных мер регуляризации или ранней остановки. На графиках метрик (Accuracy, F1 и др.) `Lion` и `AdamW` показывают хорошую сходимость. Кривые для `AdaHessian` и `AdaBelief` выглядят более зашумленными, особенно на валидационной выборке. Из графиков также можно предположить, что если бы обучение для `Lion` было прекращено раньше (например, на 5-7 эпохе), итоговое значение потерь на валидации могло бы быть ниже. Анализ матриц ошибок (рис. 6) показывает, что для всех оптимизаторов остается проблемным различение некоторых классов, например, класса 3 (кот) и класса 5 (собака). `Lion`, по-видимому, лучше других справляется с различением класса 3 от класса 5, но ошибочно относит объекты класса 5 к классу 3. Обратная ситуация наблюдается для `AdaHessian`. `AdamW` может рассматриваться как компромиссный вариант, хотя он чаще других ошибочно относит объекты к классу 2 (птица). ROC-кривые (рис. 7) имеют схожий вид для всех оптимизаторов, что указывает на сопоставимую общую разделяющую способность моделей.

4.5. Ответы на исследовательские вопросы и обсуждение результатов

На основе проведенных экспериментов можно сформулировать ответы на поставленные исследовательские вопросы.

RQ1: Какой из исследуемых алгоритмов оптимизации демонстрирует наилучшую производительность (качество модели) на различных типах задач при использовании как подобранных, так и стандартных гиперпараметров? Производительность оптимизаторов варьировалась в зависимости от задачи и использования подобранных/стандартных параметров:

- **Задача регрессии (California Housing):** С подобранными параметрами `AdaHessian` и `AdaBound` показали лучшие результаты по R^2 Score и MAPE соответственно (таблицы 2, 3). При сравнении на тестовой выборке (таблица 4), `AdaHessian(tuned)` и `AdaBound(tuned)` сохранили преимущество или показали сопоставимые результаты с лучшими. `Lion(default)` и `AdaBelief(default)` были лучше своих подобранных версий.
- **Задача классификации (Covertypes, 10%):** При подборе `Lion` показал наилучшую Accuracy (таблица 6). При обучении в течение 50 эпох (таблица 7), `AdaBelief(tuned)` продемонстрировал наивысшую Accuracy/F1, немного опередив `Lion(tuned)`. Большинство подобранных конфигураций превзошли стандартные.
- **Задача классификации изображений (CIFAR-10, ResNet-18):** При коротком подборе (5 эпох, таблица 8) `Lion` лидировал по Accuracy. При более длительном обучении (20 эпох, таблица 9) `AdaBelief(tuned)` показал наилучшие метрики качества (Accuracy/F1, Loss), подвергаясь менее всего переобучению.

В целом, нельзя выделить конкретно один оптимизатор, который бы доминировал на всех задачах. `AdaBelief(tuned)` и `Lion(tuned)` часто показывали сильные результаты, особенно на задачах классификации.

RQ2: Насколько существенна разница во времени обучения между исследуемыми оптимизаторами при достижении сопоставимого качества модели на различных задачах? Разница во времени обучения была существенной, особенно для `AdaHessian`, который стабильно требовал значительно больше времени на всех задачах (например, в 2.4-3 раза дольше других на CIFAR-10).

- На задаче регрессии (таблица 4) `AdamW` и `Lion` были самыми быстрыми (около 31-37 с), `AdaHessian` — самым медленным (46-48 с).
- На задаче классификации `Coverttype` (таблица 7), `Lion(tuned)` и `AdamW(tuned)` были самыми быстрыми (около 71 с), `AdaHessian` — самым медленным (84-86 с).
- На задаче классификации изображений CIFAR-10 (таблица 9), `AdamW(tuned)` и `Lion(tuned)` были самыми быстрыми (около 239 с), `AdaHessian` — значительно медленнее (около 685 с).

Таким образом, при выборе оптимизатора необходимо учитывать компромисс между качеством модели и временем обучения. `Lion` и `AdamW` часто предлагали хорошую скорость.

RQ3: Является ли подбор гиперпараметров критически важным для достижения высокой производительности для всех исследуемых оптимизаторов, или некоторые из них показывают хорошие результаты с параметрами по умолчанию? Целесообразность подбора гиперпараметров варьировалась:

- **Задача регрессии:** Подбор не всегда приводил к улучшению. Для `AdaHessian` и `AdaBound` он был полезен, тогда как для `Lion`, `AdaBelief` и `AdamW` стандартные параметры оказались лучше или сопоставимы.
- **Задача классификации (Coverttype):** Подбор гиперпараметров привел к улучшению качества моделей для всех рассмотренных оптимизаторов, особенно для `Lion` и `AdaBelief`.

- **Задача классификации изображений (CIFAR-10):** Подбор был критически важен. Все оптимизаторы показали значительное улучшение с подобранными параметрами по сравнению со стандартными, особенно **AdaBelief**.

Следовательно, для более сложных задач (классификация, классификация изображений с глубокими моделями) подбор гиперпараметров является важным этапом для раскрытия потенциала оптимизаторов. На более простых задачах или с другими моделями некоторые оптимизаторы могут хорошо работать и с параметрами по умолчанию.

4.6. Обсуждение результатов

Проведенное исследование показало, что выбор оптимального алгоритма оптимизации не является тривиальной задачей и зависит от множества факторов, включая тип задачи, архитектуру модели, объем данных и доступные вычислительные ресурсы.

Оптимизатор **AdaBelief** с подобранными параметрами часто демонстрировал одни из лучших результатов по качеству, особенно на задачах классификации (Coverttype и CIFAR-10), при сохранении конкурентоспособного времени обучения. Его способность адаптировать шаги на основе «веры» в градиент, по-видимому, хорошо себя проявляет на сложных ландшафтах потерь.

Lion также показал себя сильным конкурентом, часто обеспечивая наилучшее или близкое к наилучшему время обучения и высокое качество моделей, особенно после подбора гиперпараметров. Его простота и эффективность делают его привлекательным вариантом.

AdamW, будучи стандартным и широко используемым оптимизатором, обеспечивал высокую скорость обучения, но по качеству моделей на задачах классификации с подобранными параметрами несколько уступал **AdaBelief** и **Lion**.

AdaBound показал хорошие результаты на задаче регрессии после подбора, но на задачах классификации его производительность была средней.

AdaHessian, использующий информацию второго порядка, стабильно требовал значительно больше времени на обучение, что ограничивает его практическое применение в условиях ограниченных ресурсов, несмотря на то, что после подбора он мог показывать конкурентоспособное качество (например, на задаче регрессии).

Важным наблюдением является значимость подбора гиперпараметров, особенно для задач классификации и классификации изображений. Стандартные параметры не всегда являются оптимальными и могут приводить к существенно более низкому качеству моделей.

Анализ графиков обучения выявил такие аспекты, как переобучение (особенно на CIFAR-10 при 20 эпохах), различную стабильность кривых потерь для разных оптимизаторов и разную способность справляться с дисбалансом классов (видно по матрицам ошибок). Это подчеркивает необходимость комплексного анализа, а не только сравнения итоговых метрик.

4.7. Угрозы нарушения корректности

При интерпретации результатов данной работы следует учитывать следующие потенциальные угрозы нарушения корректности:

- **Внутренняя корректность (Internal Validity):**

- *Ограниченный поиск гиперпараметров:* Сетки поиска гиперпараметров, хотя и были достаточно широкими, не охватывают все возможное пространство. Существует вероятность, что для некоторых оптимизаторов существуют еще лучшие конфигурации за пределами исследованного диапазона.
- *Фиксированное число эпох для подбора:* Начальный подбор гиперпараметров проводился за ограниченное число эпох (5 или 26). Оптимальные параметры для более длительного обучения могли бы отличаться.
- *Случайность инициализации и разделения данных:* Несмотря на фиксацию **seed**, стохастическая природа обучения ней-

ронных сетей может вносить вариативность. Для более надежных выводов потребовалось бы многократное повторение каждого эксперимента с различными `seed` и усреднение результатов.

- **Внешняя корректность (External Validity):**

- *Специфичность наборов данных и моделей:* Результаты получены на конкретных наборах данных и архитектурах моделей. Производительность оптимизаторов может отличаться на других задачах, данных или с другими моделями.
- *Используемые реализации оптимизаторов:* Незначительные различия в сторонних реализациях оптимизаторов по сравнению с оригинальными статьями или другими популярными реализациями могли повлиять на результаты.

- **Конструктивная корректность (Construct Validity):**

- *Выбор метрик:* Хотя использовались стандартные метрики, их относительная важность может меняться в зависимости от конкретных целей применения модели.

Для минимизации некоторых из этих угроз были предприняты меры, такие как фиксация `seed`, использование стандартных процедур предобработки и аугментации, а также выбор широко известных наборов данных и моделей. Однако полное исключение всех угроз в рамках курсовой работы затруднительно.

4.8. Воспроизводимость эксперимента

Обеспечению воспроизводимости результатов уделялось значительное внимание. Как упоминалось в разделе 4.1, для каждого запуска экспериментов, включая этапы подбора гиперпараметров и финального обучения, применялась функция `setup`, которая устанавливала фиксированные значения `seed` для всех источников случайности (Python

`random`, NumPy, PyTorch CPU и PyTorch GPU). Это позволяет минимизировать вариативность, связанную со стохастическими аспектами обучения.

Заключение

В ходе выполнения данной курсовой работы был проведен сравнительный анализ современных алгоритмов оптимизации на различных задачах машинного обучения. Были исследованы теоретические основы выбранных алгоритмов, реализованы и протестированы их программные модули, а также проанализирована их производительность.

В соответствии с поставленными задачами (см. раздел 1) были получены следующие основные результаты:

1. **Проведен обзор теоретических основ и ключевых особенностей алгоритмов оптимизации AdaHessian, Lion, AdaBelief, AdamW и AdaBound.** В разделе 2.1 были рассмотрены принципы работы каждого алгоритма, их сильные и слабые стороны, а также предположения, лежащие в их основе. Также был выполнен обзор используемых технологий (раздел 2.2), включая фреймворк PyTorch, библиотеку для подбора гиперпараметров Ray Tune и инструменты для анализа и визуализации.
2. **Реализованы программные модули, необходимые для обучения нейросетевых моделей с использованием выбранных оптимизаторов.** В рамках работы была создана структура проекта (раздел 3.1), включающая модули для предобработки данных, построения моделей (SimpleRegressionModel, SimpleClassificationModel, использование ResNet-18), реализации циклов обучения и валидации, а также вспомогательные утилиты для логирования и визуализации. Были интегрированы реализации исследуемых оптимизаторов.
3. **Проведено экспериментальное обучение нескольких нейросетевых моделей на задачах регрессии, классификации и классификации изображений, применяя каждый из исследуемых оптимизаторов.** Были выполнены следующие эксперименты:

- *Задача регрессии* на датасете California Housing (раздел 4.4.1). Осуществлен подбор гиперпараметров и сравнение производительности оптимизаторов.
- *Задача классификации* на датасете Coverttype (10% выборка) (раздел 4.4.2). Проведен подбор гиперпараметров и анализ результатов при обучении в течение 26 и 50 эпох.
- *Задача классификации изображений* на датасете CIFAR-10 с моделью ResNet-18 (раздел 4.4.3). Выполнен подбор гиперпараметров при обучении в течение 5 эпох и более длительное обучение (20 эпох) для сравнения подобранных и стандартных конфигураций.

Для каждой задачи и каждого оптимизатора был осуществлен подбор оптимальных гиперпараметров с использованием Ray Tune по определенным сеткам поиска.

4. Выполнен сравнительный анализ полученных результатов обучения, оценены алгоритмы по ключевым метрикам и сформулированы практические выводы.

- Анализ результатов (разделы 4.4.1, 4.4.2, 4.4.3, а также обобщенное обсуждение в 4.5 и 4.6) показал, что ни один из оптимизаторов не является универсально лучшим для всех задач.
- AdaBelief с подобранными параметрами часто демонстрировал высокое качество на задачах классификации.
- Lion показал хорошее сочетание скорости и качества, особенно после подбора.
- AdaHessian стабильно требовал значительно большего времени на обучение.
- Подтверждена важность подбора гиперпараметров, особенно для более сложных задач и моделей, так как стандартные параметры не всегда обеспечивали оптимальную производительность.

- Была проведена визуализация процессов обучения и результатов (рис. 1, 2–4, 5–7), что позволило выявить особенности поведения различных оптимизаторов.

Таким образом, все поставленные задачи курсовой работы были выполнены. Проведенное исследование позволило получить практический опыт в применении и сравнении современных алгоритмов оптимизации, а также выявить их специфические особенности при решении различных задач машинного обучения.

Исходный код проекта, включая скрипты для проведения экспериментов, анализа данных и jupyter-ноутбуки, располагается в приватном репозитории на GitHub.

Список литературы

- [1] Yao Zhewei, Gholami Amir, Shen Sheng et al. ADAHESSIAN: An Adaptive Second Order Optimizer for Machine Learning. — 2021. — [2006.00719](#).
- [2] Zhuang Juntang, Tang Tommy, Ding Yifan et al. AdaBelief Optimizer: Adapting Stepsizes by the Belief in Observed Gradients. — 2020. — [2010.07468](#).
- [3] Luo Liangchen, Xiong Yuanhao, Liu Yan, Sun Xu. Adaptive Gradient Methods with Dynamic Bound of Learning Rate. — 2019. — [1902.09843](#).
- [4] Deep residual learning for image recognition / Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun // Proceedings of the IEEE conference on computer vision and pattern recognition. — 2016. — P. 770–778.
- [5] Dua Dheeru, Graff Casey. UCI Machine Learning Repository. — 2017. — URL: <http://archive.ics.uci.edu/ml>.
- [6] Google Brain Team. TensorBoard: TensorFlow’s Visualization Toolkit. — <https://www.tensorflow.org/tensorboard>. — 2017. — Accessed: 2025-05-14.
- [7] Learning multiple layers of features from tiny images : Rep. / University of Toronto ; Executor: Alex Krizhevsky, Geoffrey Hinton : 2009. — URL: <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>.
- [8] Loshchilov Ilya, Hutter Frank. Decoupled Weight Decay Regularization. — 2019. — [1711.05101](#).
- [9] Luo Liangchen, others. AdaBound Optimizer PyTorch Implementation (AdaBound GitHub Repository). — <https://github.com/Luolc/AdaBound>. — 2019. — Accessed: 2025-05-14.

- [10] McKinney Wes. Data Structures for Statistical Computing in Python // Proceedings of the 9th Python in Science Conference / Ed. by Stéfan van der Walt, Jarrod Millman. — 2010. — P. 56–61. — URL: <https://conference.scipy.org/proceedings/scipy2010/pdfs/mckinney.pdf>.
- [11] Novik Nikolay, Yoshiyama Kazuki, others. torch-optimizer: Collection of optimizers for PyTorch. — <https://github.com/optimizer-primary/torch-optimizer>. — 2020. — Accessed: 2025-05-14.
- [12] Pace R Kelley, Gilley Otis W. Sparse spatial autoregressions // Statistics & Probability Letters. — 1997. — Vol. 33, no. 3. — P. 291–297.
- [13] [PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation](#) / Jason Ansel, Edward Yang, Horace He et al. // Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2. — ASPLOS '24. — New York, NY, USA : Association for Computing Machinery, 2024. — P. 929–947. — URL: <https://doi.org/10.1145/3620665.3640366>.
- [14] Moritz Philipp, Nishihara Robert, Wang Stephanie et al. Ray: A Distributed Framework for Emerging AI Applications. — 2018. — [1712.05889](#).
- [15] Chen Xiangning, Liang Chen, Huang Da et al. Symbolic Discovery of Optimization Algorithms. — 2023. — [2302.06675](#).
- [16] Liaw Richard, Liang Eric, Nishihara Robert et al. Tune: A Research Platform for Distributed Model Selection and Training. — 2018. — [1807.05118](#).
- [17] Wang Phil. Lion Optimizer PyTorch Implementation (lion-pytorch). — <https://github.com/lucidrains/lion-pytorch>. — 2023. — Accessed: 2025-05-14.

- [18] Waskom Michael L. Seaborn: Statistical Data Visualization // [Journal of Open Source Software](#). — 2021. — Vol. 6, no. 60. — P. 3021. — URL: <https://doi.org/10.21105/joss.03021>.
- [19] Zhuang Juntang, others. AdaBelief Optimizer PyTorch Implementation (adabelief-pytorch). — <https://github.com/juntang-zhuang/Adabelief-Optimizer>. — 2020. — Accessed: 2025-05-14.